Blockchain Technology

Final Practical Exam Guide: BT1 to BT4
Step-by-step implementation, code explanation, and viva Q&A

Chit	Practical	What you must demonstrate
BT1	Install MetaMask, study Ether spent per transaction	Install, create wallet, send a transaction, read gas fee breakdown
BT2	Create your own wallet using MetaMask	Wallet creation, accounts, receive/send crypto, recovery phrase handling
BT3	Smart contract: bank account (deposit, withdraw, show balance)	Solidity code, deploy on a test network, run all three operations
BT4	Solidity: Student data with structures, arrays, fallback	Code, deploy on Ethereum test network, record transaction fee and gas

Read this first. (1) The numbers 3, 14, 15, 16 in your chit file are just chit serial numbers; the four practicals are BT1 to BT4. (2) MetaMask and Remix change their screens often, so menu names below may differ slightly from what you see. The concepts and the code do not change. (3) On exam day you may get no faucet access or restricted internet. Practice the **Remix VM fallback** described in Section 0 so you are not stuck.

Contents

- Section 0: Common setup, gas theory, and exam-day strategy
- Section 1: BT1 - MetaMask installation and Ether spent per transaction
- Section 2: BT2 - Creating your own wallet with MetaMask
- Section 3: BT3 - Bank account smart contract (deposit, withdraw, show balance)
- Section 4: BT4 - Student data contract (structure, array, fallback) with gas observation
- Section 5: Viva questions and answers (about 90 questions, grouped by topic)
- Section 6: Troubleshooting table and last-minute checklist

Section 0: Common Setup, Gas Theory, Exam Strategy

0.1 What you need

- A desktop browser: Chrome, Brave, Edge or Firefox (MetaMask extension supports these).
- MetaMask extension, downloaded only from **metamask.io** (fake extensions and fake sites exist).
- Remix IDE (browser-based Solidity IDE): **remix.ethereum.org**. Nothing to install.
- A test network. Use **Sepolia** (chain ID 11155111). The older Ropsten, Rinkeby, Kovan and Goerli testnets are shut down, so older textbooks and YouTube videos that use them will mislead you.
- Test ETH from a Sepolia faucet (free, no real value). Faucets change often and some need a small mainnet balance or a login; search 'Sepolia faucet' on the day.
- Block explorer for checking transactions: **sepolia.etherscan.io**.

0.2 Gas and transaction fee theory (needed for BT1 and BT4)

Every operation on Ethereum costs **gas**, a unit that measures computational work. You pay for gas in ETH. Since the London upgrade (EIP-1559, August 2021) the fee has these parts:

Term	Meaning
Gas limit	Maximum gas you allow the transaction to use. A plain ETH transfer needs exactly **21,000**. Contract calls need more.
Gas used	Gas actually consumed. Unused gas is not charged.
Base fee (per gas)	Set automatically by the network per block. It is **burned** (destroyed), not paid to anyone.
Priority fee / tip (per gas)	Extra you pay the validator to include your transaction sooner.
Max fee per gas	The most you are willing to pay per gas (base fee + tip). You are refunded the difference if the base fee is lower.
Gwei	Unit for gas prices. 1 gwei = 10^{-9} ETH = 10^{9} wei. 1 ETH = 10^{18} wei.

Formula: Transaction fee = Gas used x (Base fee + Priority fee)

Worked example: Plain transfer, gas used = 21,000, base fee = 12 gwei, tip = 1.5 gwei. Effective gas price = 13.5 gwei. Fee = 21,000 x 13.5 = 283,500 gwei = **0.0002835 ETH**. The amount of ETH you send does not change the fee. Only gas used and gas price do.

0.3 Remix VM fallback (practice this)

Remix has a built-in simulated blockchain called **Remix VM** (names like 'Remix VM (Cancun)'). It needs no wallet, no faucet and no internet after the page loads. It also shows gas figures in the console. If the examiner's lab blocks faucets or MetaMask, tell the examiner and demonstrate on Remix VM. [Likely] this is acceptable for BT3 and BT4 code demonstration, but a testnet deployment is what the chit literally asks for (BT4 says 'Deploy on Ethereum'), so try the testnet first.

0.4 Standard write-up format for each practical

- Aim
- Software/tools used
- Theory (short)
- Steps / algorithm
- Code (BT3, BT4)
- Output / observation table
- Conclusion

0.5 Exam-day plan

- Do the MetaMask setup and faucet request **first**; faucets can take minutes.

- Keep a notepad open with your wallet address, transaction hashes and gas figures. You will be asked for them.
- Never type your recovery phrase into any website or share it. Examiners sometimes test this awareness.
- Have both contracts saved as files on your pen drive or cloud notes in case you need to re-paste them.

Section 1: BT1 - Install MetaMask and Study Ether Spent per Transaction

Aim: To install MetaMask, set up a wallet, perform transactions on a test network, and study the Ether (gas fee) spent per transaction.

1.1 Theory in brief

MetaMask is a non-custodial wallet delivered as a browser extension and mobile app. It stores your keys encrypted on your device, signs transactions, and injects an Ethereum provider (`window.ethereum`) into web pages so decentralised apps can ask for your signature.

1.2 Step-by-step

1. Open your browser and go to **metamask.io**. Click **Download**, choose your browser. This opens the browser's extension store.
2. Click **Add to Chrome** (or Add to Brave/Edge/Firefox), then **Add extension**. Pin the fox icon from the puzzle-piece menu so it stays visible.
3. On the welcome screen choose **Create a new wallet** (do not choose import unless you already own one). Accept the terms.
4. Set a **password** (minimum 8 characters). This password only unlocks MetaMask on this device.
5. Secure the wallet: MetaMask shows a **Secret Recovery Phrase (SRP)** of 12 words. Write it on paper in order. Confirm it by selecting the requested words. Anyone with this phrase controls your funds.
6. Enable test networks: click the network selector at the top, turn on **Show test networks** (in some versions: Settings > Advanced > Show test networks). Select **Sepolia**.
7. Copy your account address (click the address under the account name). Open a Sepolia faucet, paste the address, request test ETH. Wait until the balance shows in MetaMask (usually 0.05 to 0.5 SepoliaETH).
8. Create a second account: click the account name > **Add account or hardware wallet** > **Ethereum account** > name it 'Account 2'. Copy its address.
9. From Account 1 click **Send**, choose Account 2 as recipient (or paste its address), enter **0.001** ETH, click **Next**.
10. **Study the fee screen.** Note the estimated gas fee. Choose the speed (Low / Market / Aggressive) or click **Advanced** to see Max base fee, Priority fee and Gas limit (shows 21000). Click **Confirm**.
11. Open the **Activity** tab, click the transaction, and choose **View on block explorer** (Sepolia Etherscan). Record the details in the table below.
12. Repeat with: (a) 0.002 ETH at the same speed, (b) 0.001 ETH at Aggressive priority. Compare fees.

1.3 What to read on Etherscan

Field on Etherscan	Meaning
Transaction Hash	Unique ID of the transaction
Status / Block	Success or Fail; block number that included it
From / To / Value	Sender, receiver, amount of ETH moved
Transaction Fee	Total ETH paid = Gas used x effective gas price
Gas Price	Effective price per gas in ETH and gwei
Gas Limit & Usage	Limit you set, gas used, and percentage used (21,000 / 21,000 = 100% for a plain transfer)
Gas Fees: Base / Max / Max Priority	Base fee of that block, your cap, your tip
Burnt Fees	Base fee x gas used, destroyed by the protocol

1.4 Observation table (sample only: replace with your own values)

Tx	Amount sent	Speed	Gas limit	Gas used	Gas price (gwei)	Fee (ETH)
1	0.001	Market	21000	21000	1.5	0.0000315
2	0.002	Market	21000	21000	1.5	0.0000315
3	0.001	Aggressive	21000	21000	2.1	0.0000441

Check of row 1: 21,000 x 1.5 gwei = 31,500 gwei = 0.0000315 ETH. On Sepolia the gas price is usually very low (often a few gwei or less), so your fees will look tiny. On mainnet the same transfer typically costs far more, depending on congestion.

1.5 Conclusion to write

A plain ETH transfer consumed 21,000 gas in every case. The transaction fee did **not** change when the amount changed (0.001 vs 0.002 ETH); it changed only with the gas price (priority fee). The base fee portion was burned and the tip went to the validator.

Section 2: BT2 - Create Your Own Wallet Using MetaMask for Crypto Transactions

Aim: To create a cryptocurrency wallet using MetaMask and perform crypto transactions (send and receive) on a test network.

2.1 Theory in brief

- A wallet does not store coins. Coins live on the blockchain. The wallet stores **private keys** that prove ownership and sign transactions.
- **Private key** (256-bit secret) > **public key** (via elliptic curve secp256k1) > **address** (last 20 bytes of the Keccak-256 hash of the public key, shown as 0x followed by 40 hex characters).
- MetaMask is a **hierarchical deterministic (HD)** wallet. One 12-word **Secret Recovery Phrase** (BIP-39) generates all accounts through the derivation path m/44'/60'/0'/0/n (BIP-32/44). Restoring the phrase restores every derived account.
- **Non-custodial:** you hold the keys; MetaMask the company cannot recover them. **Custodial** (like an exchange) holds keys for you.

2.2 Step-by-step

1. Install MetaMask and create a wallet exactly as in BT1 steps 1 to 5 (if already done, show the existing wallet and explain each screen).
2. Switch to the **Sepolia** test network and show the **account address**. Explain its format (0x + 40 hex characters).
3. Rename the account: account menu > three dots > **Account details** > edit name (e.g., 'Student Wallet').
4. Fund the wallet with test ETH from a faucet. Show the balance updating.
5. Add a second account (**Add account** > **Ethereum account**). This is a new key pair derived from the same recovery phrase.
6. **Send:** Account 1 > Send > select Account 2 > amount 0.005 > Confirm. Wait for the 'Confirmed' status in Activity.
7. **Receive:** switch to Account 2 and show the increased balance. Show Account 2's QR code/address as the 'receive' mechanism.
8. Open both transactions on **Sepolia Etherscan** and show the From, To, Value, and Fee.
9. Demonstrate wallet lock and unlock: lock MetaMask from the account menu, then unlock with the password.
10. Explain recovery (do **not** reveal your real phrase): Settings > Security & Privacy > Reveal Secret Recovery Phrase asks for a password. On a new browser, 'Import an existing wallet' with the 12 words restores all accounts.

11. Optional extras if time allows: **Import account** using a private key (this account is NOT covered by the recovery phrase), and add a custom network using chain ID 11155111.

> **Security points examiners like:** never share the recovery phrase or private key; MetaMask support will never ask for it; store the phrase offline; test ETH has no monetary value but the habit of protecting keys matters; beware fake MetaMask extensions and phishing popups.

2.3 Observation

Item	Your value
Account 1 address	0x... (write yours)
Account 2 address	0x... (write yours)
Network	Sepolia (chain ID 11155111)
Transfer	Account 1 to Account 2, 0.005 ETH
Tx hash / fee	(write yours)

2.4 Conclusion to write

A MetaMask wallet was created; two accounts were derived from one recovery phrase; test ETH was sent and received; transactions were verified on a block explorer.

Section 3: BT3 - Smart Contract for Customer Bank Account

Aim: To write, compile and deploy a smart contract on a test network that supports (1) deposit money, (2) withdraw money, (3) show balance.

3.1 Solidity code (Bank.sol)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract Bank {
    // maps each customer's address to his/her balance (stored in wei)
    mapping(address => uint256) private balances;

    event Deposited(address indexed customer, uint256 amount);
    event Withdrawn(address indexed customer, uint256 amount);

    // 1. Deposit money: the customer sends ETH along with the call
    function deposit() public payable {
        require(msg.value > 0, "Deposit must be greater than 0");
        balances[msg.sender] += msg.value;
        emit Deposited(msg.sender, msg.value);
    }

    // 2. Withdraw money: amount is given in wei
    function withdraw(uint256 amount) public {
        require(amount > 0, "Amount must be greater than 0");
        require(balances[msg.sender] >= amount, "Insufficient balance");
        balances[msg.sender] -= amount;              // effects first
        (bool sent, ) = payable(msg.sender).call{value: amount}("");
        require(sent, "Transfer failed");            // interaction last
        emit Withdrawn(msg.sender, amount);
    }

    // 3. Show balance of the caller
    function showBalance() public view returns (uint256) {
        return balances[msg.sender];
    }

    // Extra: total ETH held by the contract
    function getContractBalance() public view returns (uint256) {
        return address(this).balance;
```

```
    }
}
```

3.2 Code explanation (line by line)

Code	Explanation
`// SPDX-License-Identifier: MIT`	License comment. The compiler warns if it is missing.
`pragma solidity ^0.8.0;`	Contract compiles with compiler 0.8.0 or any later 0.8.x. Version 0.8 gives automatic overflow/underflow checks.
`contract Bank { ... }`	Defines the smart contract (similar to a class).
`mapping(address => uint256) private balances;`	Key-value store: customer address to balance in wei. `private` only restricts other contracts' access; the data is still readable on-chain.
`event Deposited / Withdrawn`	Logs stored in the transaction receipt. Front-ends and Etherscan read them. `indexed` makes the address searchable.
`function deposit() public payable`	`payable` allows the function to receive ETH. The ETH is credited to the contract automatically.
`msg.value`	Amount of wei sent with the call.
`msg.sender`	Address that called the function (the customer).
`require(cond, "msg")`	If the condition is false, the transaction reverts: state changes are undone and the message is returned.
`balances[msg.sender] += msg.value;`	Increases the caller's recorded balance.
`function withdraw(uint256 amount)`	Not payable because it sends ETH out rather than receiving it.
`balances[msg.sender] -= amount;` before sending	**Checks-Effects-Interactions** pattern: update state first, then send ETH. This prevents reentrancy attacks.
`payable(msg.sender).call{value: amount}("")`	Low-level call that forwards ETH to the caller and returns (success, data). Recommended over `transfer` because `transfer` forwards only 2300 gas.
`function showBalance() public view returns (uint256)`	`view` means it reads but does not modify state, so calling it from outside costs no gas.
`address(this).balance`	Real ETH balance held by the contract.

3.3 Step-by-step: compile and deploy using Remix

1. Open **remix.ethereum.org**. In File Explorer, open the `contracts` folder, click **Create new file**, name it `Bank.sol`, paste the code.
2. Open the **Solidity Compiler** tab. Choose a compiler version 0.8.x (for example 0.8.20 or later) and click **Compile Bank.sol**. A green tick appears on the compiler icon.
3. Open the **Deploy & Run Transactions** tab. For a dry run set ENVIRONMENT to **Remix VM**. For the actual exam select **Injected Provider - MetaMask**.
4. MetaMask asks to connect: select your account and approve. Make sure MetaMask is on **Sepolia**. Remix shows 'Sepolia (11155111) network' under the environment.
5. CONTRACT dropdown shows **Bank**. Click the orange **Deploy** button. MetaMask pops up with the fee estimate: click **Confirm**.
6. Wait for confirmation. The contract appears under **Deployed Contracts**. Expand it and copy its address (copy icon).
7. **Deposit:** set the VALUE box above (e.g., `0.01`) and change its unit dropdown to **Ether**. Click the red **deposit** button. Confirm in MetaMask.
8. **Show balance:** click the blue **showBalance** button. Output: `10000000000000000` (wei) = 0.01 ETH.
9. **Withdraw:** set VALUE back to `0` (withdraw is not payable). In the box beside `withdraw` enter `5000000000000000` (0.005 ETH in wei). Click **transact** and confirm in MetaMask.

10. Click **showBalance** again. Output: `5000000000000000` (0.005 ETH left).
11. Paste the transaction hash into Sepolia Etherscan. Show the **Logs** tab with the Deposited/Withdrawn events and note the gas used.

3.4 Test cases to demonstrate

Test	Action	Expected result
1	Deposit with VALUE = 0	Reverts: 'Deposit must be greater than 0'
2	Deposit 0.01 ETH, call showBalance	10000000000000000
3	Withdraw 0.005 ETH (5000000000000000 wei)	Success; balance 5000000000000000
4	Withdraw more than the balance	Reverts: 'Insufficient balance'
5	Switch to Account 2, call showBalance	0 (each address has its own balance)

Tip: 1 ETH = 10^{18} wei. To avoid typing long numbers, remember: 1 finney (0.001 ETH) = 10^{15} wei; 0.01 ETH = 10^{16} wei.

3.5 Conclusion to write

The Bank smart contract was compiled with Solidity 0.8.x and deployed to the Sepolia test network through Remix and MetaMask. Deposit, withdraw and show-balance operations worked, and invalid operations were rejected by `require` checks.

Section 4: BT4 - Student Data Contract (Structure, Array, Fallback)

Aim: To write a Solidity program that stores student data using a **structure**, an **array** and a **fallback function**, deploy it on an Ethereum network, and observe the transaction fee and gas values.

4.1 Solidity code (StudentData.sol)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract StudentData {

    // Structure: a custom data type grouping related fields
    struct Student {
        uint256 id;
        string  name;
        uint8   age;
        string  course;
    }

    // Dynamic array of structures
    Student[] public students;

    uint256 public fallbackCount;      // counts how many times fallback ran

    event StudentAdded(uint256 indexed id, string name);
    event FallbackCalled(address sender, uint256 value, bytes data);
    event EtherReceived(address sender, uint256 value);

    // Add a student record
    function addStudent(
        uint256 _id,
        string memory _name,
        uint8 _age,
        string memory _course
    ) public {
        students.push(Student(_id, _name, _age, _course));
        emit StudentAdded(_id, _name);
    }

    // Read one student by array index
```

```
    function getStudent(uint256 index)
        public view
        returns (uint256, string memory, uint8, string memory)
    {
        require(index < students.length, "Index out of range");
        Student memory s = students[index];
        return (s.id, s.name, s.age, s.course);
    }

    // Number of students stored
    function getTotalStudents() public view returns (uint256) {
        return students.length;
    }

    // Fallback: runs when a call matches no function
    fallback() external payable {
        fallbackCount++;
        emit FallbackCalled(msg.sender, msg.value, msg.data);
    }

    // Receive: runs when plain ETH is sent with empty calldata
    receive() external payable {
        emit EtherReceived(msg.sender, msg.value);
    }
}
```

4.2 Code explanation

Code	Explanation
`struct Student {...}`	**Structure**: user-defined type that groups different data types (number, string, small number, string) into one record.
`Student[] public students;`	**Array**: dynamic array of `Student` structs. `public` makes Solidity auto-generate a getter `students(index)` that returns the fields (strings and numbers; not nested arrays).
`uint8 _age`	Unsigned 8-bit integer (0 to 255), enough for age and cheaper to pack.
`string memory _name`	`memory` is a temporary data location for function arguments of reference types like string. Required keyword for strings in function parameters.
`students.push(Student(...))`	Creates a struct instance and appends it to the array. This writes to **storage**, the most expensive operation (a fresh 32-byte storage slot costs roughly 20,000 gas to write).
`emit StudentAdded(...)`	Logs an event; cheap way to record data for off-chain apps.
`function getStudent(...) view returns (...)`	Reads a struct from storage into memory and returns multiple values. `require` blocks out-of-range indexes.
`students.length`	Number of elements currently in the dynamic array.
`fallback() external payable`	**Fallback function** (Solidity 0.6+ syntax, no `function` keyword, no name, no arguments). Executes when (a) someone calls a function that does not exist, or (b) ETH arrives with non-empty data and no matching function. `payable` lets it accept ETH.
`receive() external payable`	Executes when ETH is sent with **empty** calldata (plain transfer). If `receive` is absent, a payable fallback handles plain ETH too.
`msg.data`	Full calldata of the call. Useful to show what triggered the fallback.

Dispatch rule (examiners love this)

Incoming call	Which function runs?
Calldata matches a function selector	That function
Calldata does not match, no ETH or ETH with data	`fallback()` (if absent: revert)
Calldata empty and ETH > 0	`receive()` (if absent: payable `fallback()`; if both absent: revert)
Calldata empty and ETH = 0	`receive()` if present, otherwise `fallback()`

4.3 Step-by-step: compile, deploy, test

1. In Remix create `StudentData.sol` in `contracts`, paste the code.
2. Compiler tab: version 0.8.x, click **Compile**. Fix any red errors (usually version mismatch).
3. Deploy tab: ENVIRONMENT = **Injected Provider - MetaMask** (Sepolia). CONTRACT = **StudentData**. Click **Deploy**, confirm in MetaMask. **Note the deployment gas and fee** (see 4.4).
4. Expand the deployed contract. Next to **addStudent** enter: `101`, `"Rahul"`, `20`, `"Computer"` (strings in double quotes, separated by commas) and click **transact**. Confirm in MetaMask.
5. Add a second student: `102`, `"Sneha"`, `21`, `"IT"`.
6. Click **getTotalStudents**: output `2`. Click **getStudent** with `0`: output id 101, Rahul, 20, Computer. Also try the `students` getter with `1`.
7. **Trigger the fallback:** scroll to the bottom of the deployed contract to **Low level interactions**. In the **CALLDATA** box type `0x12345678` (a function selector that does not exist). Click **Transact**. Confirm in MetaMask.
8. Click **fallbackCount**: it shows `1`. Open the transaction logs to see the `FallbackCalled` event. Optionally set VALUE to 0.001 ETH with empty calldata and Transact to trigger `receive()`.
9. Open each transaction hash on Sepolia Etherscan and record gas used, gas price and transaction fee.

4.4 Where to read gas and fee

- **Remix console** (bottom panel) after each transaction shows: status, `gas` (gas limit), `transaction cost` (total gas used including the 21,000 base) and `execution cost` (gas consumed by the EVM code).
- **MetaMask** Activity > transaction > shows the network fee.
- **Etherscan:** 'Gas Used by Transaction', 'Gas Price', 'Transaction Fee', 'Burnt Fees'.

4.5 Observation table (fill with your own values)

Transaction	Gas used	Gas price (gwei)	Fee = gas x price (ETH)	Comment
Deploy StudentData	(yours)	(yours)	(yours)	Largest: stores contract bytecode
addStudent #1	(yours)	(yours)	(yours)	Writes new storage slots (strings and numbers)
addStudent #2	(yours)	(yours)	(yours)	Similar cost to #1
Fallback call	(yours)	(yours)	(yours)	Cheaper: one counter increment and an event

Expected pattern [Likely]: deployment uses by far the most gas; each `addStudent` costs on the order of 100,000 gas or more because it writes several storage slots; a fallback call that increments one counter costs roughly 30,000 to 50,000 gas. **[Guessing]** on exact numbers: they depend on string length and the compiler version, so quote only what you actually measured.

Why deployment is expensive: a contract creation costs 32,000 gas for the CREATE operation, plus 200 gas for every byte of contract code stored, plus the cost of calldata and constructor execution.

4.6 Conclusion to write

A Student contract using a struct, a dynamic array and a fallback function was deployed to the Sepolia testnet. Student records were stored and read back, the fallback was triggered with unknown calldata, and gas and fee were observed: deployment cost the most gas, state-changing calls cost more than read-only calls (which cost nothing when called externally).

Section 5: Viva Questions and Answers

Learn the **bold ideas**. Examiners usually ask one or two basics, then go into what you just demonstrated.

A. Blockchain fundamentals

Q1. What is a blockchain?

A distributed, append-only ledger. Transactions are grouped into blocks; each block stores the hash of the previous block, so blocks form a chain. Many nodes keep copies and agree on the state through a **consensus mechanism**.

Q2. What does a block contain?

A header (previous block hash, Merkle root of transactions, timestamp, and consensus-specific fields) and a body (the list of transactions).

Q3. Why is a blockchain tamper-resistant?

Changing data in one block changes its hash, which breaks the link stored in the next block. An attacker would have to redo the work or stake behind all later blocks and beat the honest majority.

Q4. What is a cryptographic hash function? Which one does Ethereum use?

A function that maps any input to a fixed-size output. It is deterministic, fast, one-way and collision resistant, and a tiny input change completely changes the output (avalanche effect). Ethereum uses **Keccak-256** (Bitcoin uses SHA-256).

Q5. What is a Merkle tree?

A binary tree of hashes where leaves are transaction hashes and the root summarises all of them. It lets a light client verify that a transaction is included without downloading the full block.

Q6. What is consensus? Name types.

A rule by which nodes agree on the next valid block. Examples: **Proof of Work** (Bitcoin, old Ethereum), **Proof of Stake** (Ethereum now), Proof of Authority, PBFT.

Q7. Which consensus does Ethereum use and since when?

Proof of Stake since **The Merge on 15 September 2022**. Before that it used Proof of Work.

Q8. Difference between public, private and consortium blockchains?

Public: anyone can join and read/write (Ethereum, Bitcoin). Private: one organisation controls membership. Consortium: a group of organisations controls it (Hyperledger Fabric deployments).

Q9. What is a digital signature in blockchain?

A value created with the sender's private key over the transaction data. Anyone can verify it with the public key, proving authenticity and integrity without revealing the private key. Ethereum uses **ECDSA on secp256k1**.

Q10. What is double spending and how is it prevented?

Spending the same coin twice. It is prevented because all nodes agree on one ordered ledger through consensus, and a spent output or nonce cannot be reused.

Q11. What is a 51% attack?

An attacker controlling a majority of hash power (PoW) or stake (PoS) could reorder or censor recent transactions. It becomes economically very costly on large networks.

Q12. Bitcoin vs Ethereum?

Bitcoin is mainly a payment network with limited scripting and a UTXO model. Ethereum is a programmable platform with an account model, the **Ethereum Virtual Machine (EVM)** and smart contracts.

B. Ethereum, gas and transactions

Q1. What is the EVM?

The Ethereum Virtual Machine is the runtime in every node that executes smart contract bytecode deterministically, so all nodes reach the same result.

Q2. Ether, gwei and wei: relation?

1 ETH = 10^9 gwei = 10^{18} wei. Wei is the smallest unit. Contract balances and `msg.value` are in wei.

Q3. What is gas and why does it exist?

Gas measures computation and storage effort. It prevents infinite loops and spam, because every operation costs something, and it pays validators for their work.

Q4. Gas limit vs gas used vs gas price?

Gas limit: maximum you allow. Gas used: what was actually consumed. Gas price: ETH (in gwei) paid per unit of gas. Fee = gas used x gas price.

Q5. Explain EIP-1559.

Introduced in the London upgrade (August 2021). Fee = base fee (set by the protocol, **burned**) + priority fee (tip to the validator). The base fee rises when blocks are more than half full and falls when less than half full, by up to 12.5% per block. Users set a **max fee** and unused amount is refunded.

Q6. Why does a simple ETH transfer cost 21,000 gas?

It is the fixed intrinsic gas of any transaction (signature verification, nonce handling, state update). Contract interactions add more on top of it.

Q7. Does the amount of ETH sent change the fee?

No. The fee depends on gas used and gas price only (BT1 observation).

Q8. What happens if gas runs out mid-transaction?

The transaction fails ('out of gas'), all state changes are rolled back, and the gas consumed up to that point is **not** refunded.

Q9. Does a failed transaction cost money?

Yes. Validators did the work, so you pay for the gas used up to the failure.

Q10. Why is deploying a contract more expensive than calling it?

Deployment stores bytecode on-chain (200 gas per byte) plus a 32,000 gas creation cost, calldata cost, and constructor execution.

Q11. Why is writing to storage expensive and reading cheap?

Storage is permanent and replicated on every node. Writing a new slot costs roughly 20,000 gas; reads cost far less. Memory and stack operations are cheapest.

Q12. What is a nonce in a transaction?

A per-account counter of transactions sent. It orders transactions and prevents replay. A stuck transaction can be replaced by sending another with the same nonce and higher fee.

Q13. EOA vs contract account?

An externally owned account is controlled by a private key (your MetaMask account). A contract account is controlled by its code and has no private key.

Q14. What is a testnet? Why use Sepolia?

A network that mimics mainnet but uses valueless test ETH, so developers can test freely. Sepolia is a current, actively maintained testnet; older ones such as Ropsten, Rinkeby and Goerli are deprecated.

Q15. What is a faucet?

A service that gives free test ETH to a given address for testing.

Q16. What is a block explorer?

A website (Etherscan) that lets anyone search blocks, transactions, addresses and contracts on the chain.

Q17. Can a view function be called for free?

Called from outside the blockchain (Remix, web app) it runs locally on the node and costs no gas. Called from inside a transaction by another function it consumes gas.

C. MetaMask and wallets

Q1. What is MetaMask?

A non-custodial software wallet (browser extension and mobile app) that manages keys, signs transactions and connects websites/dApps to Ethereum networks.

Q2. Custodial vs non-custodial wallet?

Custodial: a third party (exchange) holds your keys. Non-custodial: you hold them, so you have full control and full responsibility.

Q3. What is the Secret Recovery Phrase?

A 12-word mnemonic (BIP-39) that generates all your private keys through a deterministic path. Anyone who has it controls every account derived from it.

Q4. What if you forget the MetaMask password? What if you lose the phrase?

Password forgotten: reset the wallet and restore using the recovery phrase. Phrase lost (and no wallet access): funds are **unrecoverable**; nobody, including MetaMask, can restore them.

Q5. Relation between private key, public key and address?

Private key (random 256-bit number) gives the public key by elliptic-curve multiplication; the address is the last 20 bytes of the Keccak-256 hash of the public key.

Q6. Does the wallet store your coins?

No. Coins exist on the blockchain; the wallet stores keys that authorise spending.

Q7. What is an HD wallet?

Hierarchical deterministic wallet: one seed produces a tree of key pairs (BIP-32/44), so one backup restores many accounts.

Q8. Create account vs Import account in MetaMask?

Create derives a new account from your recovery phrase. Import adds an account from a separate private key; it is **not** recovered by the phrase, so back it up separately.

Q9. Mainnet vs testnet in MetaMask?

Mainnet (chain ID 1) uses real ETH. Test networks like Sepolia (chain ID 11155111) use test ETH of no market value. Always check the network before confirming.

Q10. Signing a message vs sending a transaction?

Signing proves you control a key and does not change the blockchain or cost gas. Sending a transaction changes state and costs gas.

Q11. What does 'Injected Provider - MetaMask' mean in Remix?

Remix uses the `window.ethereum` provider that MetaMask injects into the page, so deployments and calls are signed by your MetaMask account and sent to the selected network.

Q12. How do you avoid phishing?

Install only from metamask.io, never share the phrase or private key, check URLs, read signature requests, and use a separate wallet for experiments.

D. Solidity and smart contracts

Q1. What is a smart contract?

Code stored and executed on a blockchain at a fixed address. It runs exactly as written when called, with no central operator.

Q2. What is Solidity?

A statically typed, contract-oriented, high-level language for the EVM. It compiles to bytecode and an ABI.

Q3. What are bytecode and ABI?

Bytecode is the compiled machine code stored on-chain. ABI (Application Binary Interface) is a JSON description of functions and events that tells tools how to encode calls and decode results.

Q4. What is Remix IDE?

A browser-based IDE to write, compile, deploy and debug Solidity contracts, with a built-in test blockchain (Remix VM).

Q5. Explain `pragma solidity ^0.8.0;`

Declares the compiler versions the code supports: 0.8.0 and above but below 0.9.0. It avoids compiling with an incompatible compiler.

Q6. Visibility specifiers?

`public` (anyone), `external` (only from outside), `internal` (this contract and children), `private` (this contract only). Note that 'private' does not hide data on-chain.

Q7. `view` vs `pure`?

`view` reads state but does not change it. `pure` neither reads nor changes state.

Q8. What does `payable` do?

Allows a function (or address) to receive ETH. Without it, sending ETH to the function reverts.

Q9. `msg.sender`, `msg.value`, `msg.data`?

Caller's address, wei sent with the call, and the raw calldata.

Q10. `require` vs `assert` vs `revert`?

`require` validates inputs/conditions and reverts with a message. `revert` reverts explicitly (often with custom errors). `assert` checks invariants that should never fail.

Q11. What are events?

Logs written to the transaction receipt. They are cheaper than storage and are read off-chain by applications. `indexed` parameters are searchable.

Q12. Data locations: storage, memory, calldata?

Storage: permanent state on-chain (expensive). Memory: temporary during a call (cheap). Calldata: read-only input data of external calls (cheapest).

Q13. Mapping vs array?

Mapping: key-value lookup with constant-cost access, cannot be iterated or have its length read. Array: ordered list with an index and `length`, can be iterated (but loops cost gas).

Q14. Can you edit a smart contract after deployment?

No, deployed code is immutable. You deploy a new contract or use upgradeable proxy patterns.

Q15. What is reentrancy?

An attack where an external call lets the receiver call back into your function before state is updated, draining funds (the 2016 DAO hack). Prevent it with Checks-Effects-Interactions and reentrancy guards.

Q16. What changed in Solidity 0.8 regarding arithmetic?

Overflow and underflow now revert automatically; earlier versions wrapped silently and needed SafeMath.

E. BT3 Bank contract

Q1. Why did you use a mapping for balances?

To store a separate balance for each customer address with constant-time lookup.

Q2. Why is balance stored in wei, not ether?

The EVM has no decimals; `msg.value` and all balances are integers in wei. Conversion to ETH is done off-chain.

Q3. Why is `deposit` payable but `withdraw` not?

`deposit` receives ETH from the caller. `withdraw` sends ETH out, taking only an amount parameter.

Q4. Why do you reduce the balance before sending ETH?

To follow Checks-Effects-Interactions. If the balance were reduced after, a malicious contract could re-enter `withdraw` and drain funds.

Q5. `call` vs `transfer` vs `send`?

`transfer` reverts on failure and forwards 2300 gas; `send` returns false on failure with 2300 gas; `call` forwards all gas and returns a success flag. `call` is the currently recommended approach because the 2300 gas limit can break with contract receivers.

Q6. Is `private` balance really private?

No. Anyone can read contract storage through the node. `private` only blocks other contracts from accessing it via Solidity.

Q7. Why is `showBalance` free?

It is a `view` function called from outside, so it executes locally without a transaction.

Q8. What happens if you send ETH directly to the Bank address?

It reverts because the contract has no `receive` or payable `fallback`. ETH must go through `deposit`.

Q9. How would you improve the contract?

Add an owner and access control, a transfer function between customers, a minimum balance, a reentrancy guard, a deposit-history array, interest logic, and unit tests.

Q10. What if a user withdraws without depositing?

The `require(balances[msg.sender] >= amount)` check fails and the transaction reverts with 'Insufficient balance'.

F. BT4 Student contract (struct, array, fallback)

Q1. What is a structure in Solidity?

A user-defined type that groups variables of different types, e.g., `struct Student {uint256 id; string name; ...}`.

Q2. Dynamic vs fixed-size arrays?

Fixed: size set at compile time (`uint[5]`). Dynamic: size can grow with `push` (`Student[]`). Storage dynamic arrays support `push`, `pop` and `length`.

Q3. What is a fallback function?

A special unnamed function `fallback() external [payable]` executed when a call matches no function signature, or when ETH is sent with data and no function matches.

Q4. Fallback vs receive?

`receive()` handles plain ETH transfers with empty calldata. `fallback()` handles unknown function calls (and plain ETH if `receive` is missing, provided it is payable).

Q5. What if fallback is not payable and someone sends ETH with unknown data?

The transaction reverts and the ETH is returned.

Q6. Why use a fallback function in practice?

To accept ETH, to log or handle unexpected calls, and in proxy contracts to forward calls to an implementation contract using delegatecall.

Q7. How did you trigger the fallback in Remix?

Using Low level interactions: entered calldata such as `0x12345678` that matches no function and clicked Transact. The `fallbackCount` variable and the event confirmed it.

Q8. Why `string memory` in function parameters?

Strings are reference types, so Solidity needs the data location. For public/external function parameters `memory` or `calldata` is required.

Q9. How do you get a student's data?

Call `getStudent(index)` or the auto-generated getter `students(index)`. The index is the array position, not the student id.

Q10. How can you delete a student?

`delete students[i]` only resets fields to defaults. To really remove one, swap the last element into position i and call `students.pop()`. This changes order.

Q11. What did you observe about gas?

Deployment used the most gas. `addStudent` used much more than a read-only call because it writes to storage. The fallback call used less than `addStudent`. Fee = gas used x gas price.

Q12. Why does a longer name cost more gas?

Strings longer than 31 bytes take extra storage slots, and calldata costs per byte, so cost grows with length.

Q13. Why is looping through a large array risky?

Each iteration costs gas; with many elements a transaction can exceed the block gas limit and fail. Use mappings or pagination.

G. Scenario and curveball questions

Q1. Your transaction is stuck pending. Why and what can you do?

The gas price is lower than current network demand. In MetaMask use **Speed up** (resend with higher fee, same nonce) or **Cancel** (send 0 ETH to yourself with the same nonce and higher fee).

Q2. Remix says 'Gas estimation failed' or 'transaction will likely fail'. Meaning?

The call would probably revert, for example a failed `require`, a payable function with VALUE 0, or a wrong argument. Fix the input before forcing the transaction.

Q3. MetaMask says 'Insufficient funds'. Why?

Balance is lower than value plus maximum gas fee. Request more test ETH or reduce the VALUE.

Q4. The Deploy button does nothing / compilation errors appear. What do you check?

The compiler version matches the pragma, no syntax errors, the correct contract is selected, MetaMask is unlocked and connected.

Q5. You deployed to Remix VM but cannot see it in Etherscan. Why?

Remix VM is a local simulated chain in your browser, not a public network. Deploy via Injected Provider on Sepolia to get a public transaction.

Q6. Where is your contract stored after deployment?

Its bytecode and state are stored on the blockchain at the contract address, replicated across all nodes. Remix only keeps the source file locally.

Q7. Can you reverse a confirmed transaction?

No. Confirmed transactions are final; a mistaken send can only be fixed by the recipient sending it back.

Section 6: Troubleshooting and Last-Minute Checklist

Problem	Likely cause	Fix
No 'Sepolia' in MetaMask	Test networks hidden	Network selector > Show test networks (or Settings > Advanced)
Faucet gives nothing	Rate limit, login or mainnet-balance requirement	Try another faucet; wait; use another address; fall back to Remix VM
Remix compile error about version	Pragma vs compiler mismatch	Select a 0.8.x compiler in the Solidity Compiler tab
Deposit reverts	VALUE = 0, or unit left as wei	Set VALUE and unit (Ether/Gwei/Wei) before clicking deposit
Withdraw reverts	Amount greater than balance, or amount typed in ETH	Enter amount in wei; check showBalance first
withdraw fails with VALUE set	Non-payable function cannot take ETH	Set VALUE to 0 before clicking withdraw
Strings rejected in addStudent	Missing quotes	Type strings in double quotes: `"Rahul"`
Fallback not triggered	Calldata matched a real function or empty data	Use calldata like `0x12345678` in Low level interactions
Remix not asking MetaMask	Environment still set to Remix VM	Select Injected Provider - MetaMask
Tx fee in MetaMask differs from Etherscan	MetaMask shows an estimate (max); Etherscan shows actual	Quote Etherscan's actual Transaction Fee

Final checklist

- MetaMask installed, wallet created, recovery phrase safely noted (not shown to anyone).
- Sepolia selected, balance above zero, second account created.
- At least one plain transfer done; fee, gas used and gas price noted for BT1.
- Bank.sol and StudentData.sol saved in a file you can paste quickly.
- Both contracts compiled with the same 0.8.x compiler, deployed, and each function tried once.
- Transaction hashes and gas values written down for BT4.
- You can explain: gas, EIP-1559 fee formula, struct, array, fallback vs receive, payable, view, require, events, msg.sender.